
ARI: Measuring Whether a Hosted Model Returns the Same Output Twice

The SEMQ Group

Abstract

Sending the same input to the same hosted model does not guarantee the same output, and providers do not report how often outputs change. We present the AI Reproducibility Index (ARI), a black-box protocol that measures output reproducibility under conditions any user can induce: a fresh process, concurrent load, and a gap of more than 24 hours. ARI-R measures embedding models by encoding each vector with a public, calibration-free quantizer and reporting two numbers per condition: the exact-match rate and a dimension-normalized coordinate change rate. ARI-D measures generation endpoints by byte-identical repeats. Against ground-truth vectors from eight encoders, the quantizer ignores floating-point jitter that changes the bytes of 2–100% of vectors, changes the codes of 99–100% of vectors under precision changes, and its change rate does not depend on dimension between 384 and 1,024. Across five embedding APIs, exact-match agreement ranges from 0.169[†] to 1.000[‡]; across five generation endpoints, byte-identical repeat rates range from 0.107 to 1.000, and a gateway serving the vendor’s model snapshot is less stable than the vendor API. Self-hosted fp32 inference is bit-reproducible across processes and threads and agrees at ≥ 0.996 across GPU models, while enabling TF32 changes 54% of document codes with no detectable change in Recall@10. ARI gives practitioners a fixed procedure, signed reports, and two numbers that answer how reproducible their model is.

1 Introduction

Problem.

- Same input + same hosted model $\nrightarrow$ same output: batching, kernel choice, precision tiers and backend changes alter numerics [1, 2, 8].
- Providers publish no reproducibility numbers; users cannot see backend state.
- Task metrics (Recall@10, accuracy) do not register numerical change (§4.3).

Who needs a number.

- Systems that *store* a function of the output: cache keys, dedup hashes, semantic IDs [3], vector indexes built once and queried later.
- Audits and regulated pipelines that must re-derive a past result.
- Teams choosing a provider, a precision tier or a serving stack.

Contributions.

- (1) **ARI-R**: a protocol and two metrics for embedding reproducibility — exact-match rate (HER) and a dimension-normalized change rate (ρ) — on a public, calibration-free quantizer validated against ground truth (§3).
- (2) **ARI-D**: the same conditions applied to generation endpoints, scored on byte-identical repeats, with no probe (§5).

- (3) **Measurements:** five embedding APIs, eight self-hosted encoders on CPU and three GPU architectures, five generation endpoints; plus a practitioner guide (§6).

2 The ARI protocol

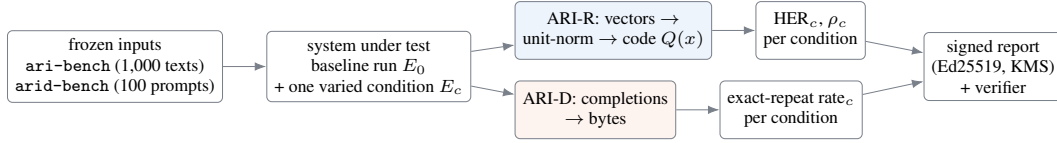


Figure 1: ARI pipeline. One baseline, one varied condition at a time, frozen hash-pinned inputs, a signed report anyone can verify without our code.

Table 1: Conditions. The core (bold) is externally inducible on any hosted API.

condition	what changes	question it answers	who can run it
same	nothing (immediate repeat)	control	everyone
proc	fresh process / client	“is a restart safe?”	everyone
conc	64-way burst vs calm	“does load change my output?”	everyone
time	gap > 24 h (we use ~76 h)	“will it match next week?”	everyone
mach	host / GPU	portability	self-hosted
prec	bf16, fp16, int8, TF32	serving-tier effect	self-hosted

- One execution pair per condition; bootstrap CIs over inputs (not over serving realizations).
- Report carries: per-condition metrics, environment manifest, input content hash, audit digest of codes, signature.
- **[burst size and Mistral exception written into spec/condition-set.md]**

3 ARI-R: embedding reproducibility

3.1 The probe

- Rescale each vector to unit norm; encode with SEMQ QUANT (public SDK, `semq==1.0.0`): per coordinate, sign + magnitude bin over the fixed range $2/\sqrt{d}$; 2 bits/coordinate.
- No calibration: the range depends on d only, so any party computes identical codes from identical vectors (byte-identical across CPUs and language bindings).
- Why fixed, not calibrated: re-calibrating the old probe on a different corpus slice moved the scale by 0.3–3.6% and **changed 60–100% of codes for identical vectors** (App. A).

Why a quantizer and not a hash.

- **Ignores sub-threshold noise.** A hash flags any bit change; the code flags a change only when a coordinate crosses a bin edge (Fig. 3).
- **Locates what moved.** Codes are per-coordinate, so a diff names the coordinates and inputs that changed; a hash says only “different”.
- **Discrete stored state.** 2 bits/dim ($16\times$ smaller than fp32); a baseline can be stored, content-addressed and signed, then compared later without keeping vectors.

Table 2: What each stored artifact can answer, per 1,024-d vector.

artifact	bytes	detects change	ignores jitter	locates change
SHA-256 of fp32 bytes	32	yes, any bit	no	no
ARI-R code ($n_{\text{bins}}=2$)	256	above bin edge	yes	yes (coordinate)
fp32 vector	4,096	yes, exact	no	yes (exact value)

3.2 Metrics: HER and ρ

- HER_e = fraction of inputs whose *whole* code matches. Answers “will my stored key/ID still match?”
- ρ_c = fraction of *coordinates* whose symbol changed, averaged over inputs. Answers “how much does the output move?”
- **HER depends on dimension, ρ does not** (Fig. 2): at equal noise, HER falls from 0.64 (384-d) to 0.26–0.32 (1,024-d) while ρ stays at $1.1\text{--}1.4 \times 10^{-3}$.
- **Headline score** $ARI\text{-}R = 1 - \bar{\rho}_{\{\text{proc}, \text{conc}, \text{time}\}}$ (dimension-normalized; use to compare models). HER is reported per condition alongside.
- Limits of normalization: holds for 384–1,024-d under isotropic relative noise; **[verify at 1,536 and 3,072-d on API data]**; models with many near-zero coordinates flip signs more easily.

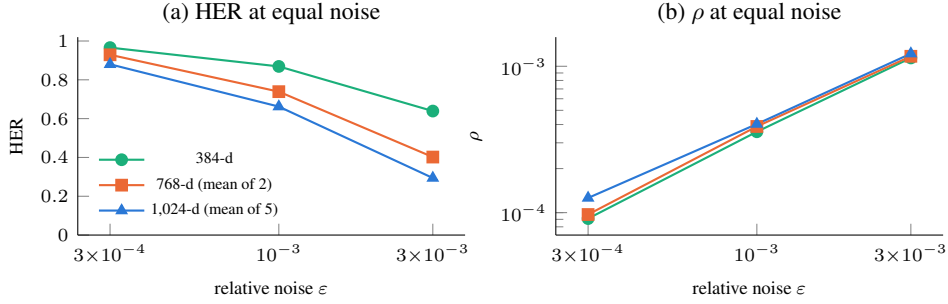


Figure 2: Same noise, eight encoders. HER falls with dimension; ρ does not. Synthetic per-coordinate relative noise on 1,000 frozen inputs, v0.2 probe.

3.3 Validation against ground truth

- Raw float32 vectors from 8 encoders, 1,000 inputs, reference fp32/CPU/4 threads/batch 32, one varied condition per fresh process; “changed” = any byte differs.
- 0 false positives: no unchanged vector ever changed code.
- Code change tracks true L2 drift (Spearman 0.88–0.95 per model).

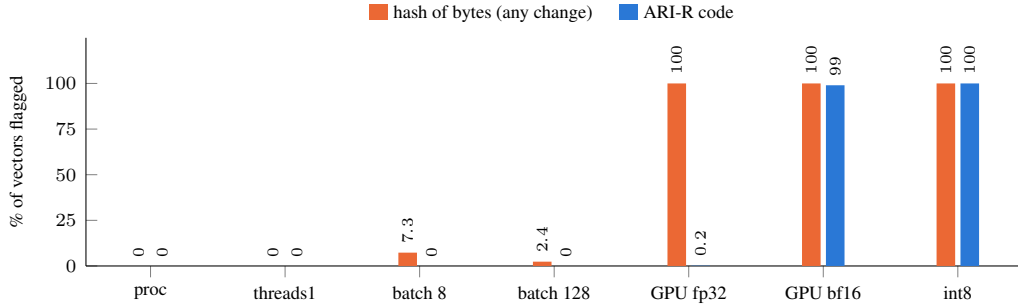


Figure 3: Selectivity, mean over 8 encoders. Batch-size and CPU→GPU fp32 jitter change bytes but not codes; precision changes are flagged. GPU = Apple M5 Max.

4 ARI-R findings

4.1 Hosted embedding APIs

- Conditions expose different failures: one provider collapses only under concurrency ($0.209^\dagger$), another only after the multi-day gap ($0.554^\dagger$); one is bit-identical in every condition.
- Drifting providers move many coordinates at small magnitude ($\sim 2,300$ of 3,072 dims at $\sim 10^{-4}$) $\Rightarrow$ backend numerics, not last-bit noise.

Table 3: Five embedding APIs, 1,000 frozen inputs. $\dagger$ = v0.1 placeholder, v0.2 rerun 2026-10-09 (adds ρ).

model	d	HER proc	HER conc	HER time	ARI-R ($1 - \bar{\rho}$)	95% CI
gemini-embedding-001	3072	1.000 $\dagger$	1.000 $\dagger$	1.000 $\dagger$	□	□
text-embedding-3-large	3072	0.862 $\dagger$	0.851 $\dagger$	0.822 $\dagger$	□	□
mistral-embed	1024	0.753 $\dagger$	0.791 $\dagger$	0.554 $\dagger$	□	□
voyage-4-large	1024	0.615 $\dagger$	0.209 $\dagger$	0.676 $\dagger$	□	□
embed-v4.0	1536	0.146 $\dagger$	0.130 $\dagger$	0.232 $\dagger$	□	□

- Rank providers only on ARI-R (ρ), never on HER. [figure: dot plot, one row per model, marks for proc/conc/time]

4.2 Self-hosted encoders

Table 4: Eight encoders (384–1,024-d), v0.2 rerun. HER per condition; GPU rows $n = 512$.

condition	setting	HER (range over 8 encoders)
proc, threads, batch	CPU fp32	1.000
proc	A10G / T4 fp32, TF32 off	1.000
mach	A10G $\leftrightarrow$ T4, fp32	0.996–1.000
proc	A10G fp32, TF32 on	0.594–0.848 (mean 0.725)
prec	bf16 / int8 (CPU)	[CPU panel; locally 0.000–0.069]
conc, time (>24h)	CPU	[2026-10-07]
proc / conc / prec	SFR-Embedding-2_R (7B, 4,096-d), L40S fp32*	1.000 / 0.998 / 0.000
batch 1 vs 32	H100	[2026-10-13]

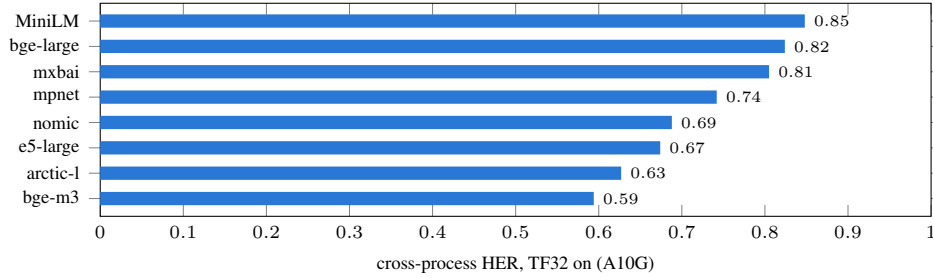


Figure 4: TF32 alone breaks cross-process reproducibility; TF32 off gives 1.000 for all eight; `use_deterministic_algorithms` has no effect (identical values).

- Pinning precision (fp32, TF32 off) $\Rightarrow$ bit-reproducible across process and threads; ≥ 0.996 across GPU models.
- Exposed axis is CPU $\leftrightarrow$ GPU and TF32; portable axis is GPU $\leftrightarrow$ GPU.
- Concurrent encoding on one shared model instance is not perfectly reproducible: SFR conc = 0.998 (2 of 1,000 inputs). *fp32 compute on bf16-loaded weights.

4.3 Task metrics do not register the change

- Numerical change is real but tiny (dot products ≈ 1.0000); Recall@10 is invariant to it, codes are not.
- Claim is *not* “retrieval misses large drift”; it is “task metrics cannot tell you whether outputs are reproducible”.

4.4 Case study: a silent precision change

- mxbai-embed-large-v1 declares `float16` in its config; transformers 5 loads the declared dtype unless told otherwise.

Table 5: BEIR SciFact (5,183 docs, 300 judged queries), all-MiniLM-L6-v2, v0.2. [rerun at $n_{\text{bins}}=2$ to match the ARI-R probe (currently 8 bins)].

condition	codes changed	$\Delta R@10$	95% CI	top-10 same
proc / threads / batch 8 / 128	0%	0.000	[0, 0]	100%
GPU fp32, TF32 off	0.1%	0.000	[0, 0]	100%
GPU fp32, TF32 on	54%	0.000	[0, 0]	96%
GPU fp16	100%	0.000	[0, 0]	93%
bf16, CPU	100%	+0.003	[0, 0.010]	45%
bf16, GPU	100%	-0.003	[-0.010, 0]	43%
int8 (CPU)	100%	-0.006	[-0.028, 0.016]	0%

- An “fp32” pipeline ran it in fp16: 12,130 distinct coordinate values in 1M (vs 1,009,495 in fp32); CPU vs GPU HER ≈ 0.5 (fp16) vs 0.999 (fp32).
- ARI surfaced it as an outlier; a library default, not the model or hardware, caused it. [one-panel figure]

5 ARI-D: generation reproducibility

- 100 frozen prompts (5 buckets), temperature 0 (+ seed where accepted), $k = 8$ repeats per condition ($k = 12$ under burst).
- Score: fraction of repeat *pairs* with byte-identical completions, averaged over prompts; bootstrap CI over prompts.
- No quantizer: bytes are what an API user observes; anyone can check it.

Table 6: Byte-identical repeat rate, 5 endpoints (dry run). Superscript = achieved burst.

endpoint	class	same	proc	conc	time	ARI-D
gemini-3.6-flash	vendor	1.000	1.000	1.000 ⁶⁴		
mistral-medium-2604	vendor	0.525	0.550	0.432 ¹⁶		
gpt-4o-mini	vendor	0.470	0.443	0.479 ⁶⁴	0.469	0.464
Llama-3.3-70B-Turbo	rehosted	0.452	0.431	0.448 ⁶⁴	0.435	0.438
gpt-4o-mini	platform	0.107	0.115	—	0.109	—

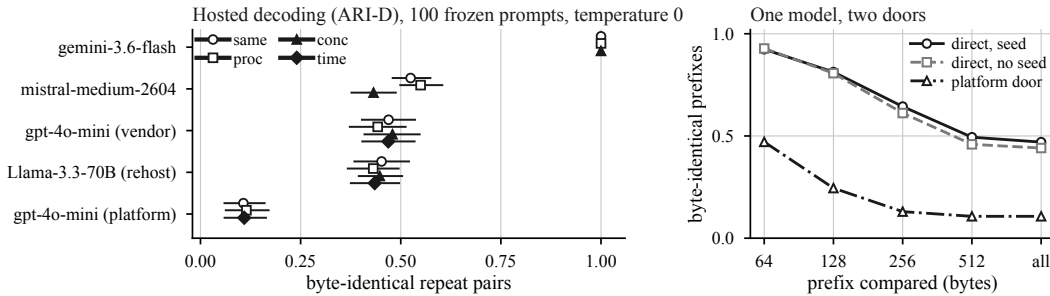


Figure 5: Left: repeat rates with 95% CIs. Right: the same model snapshot via vendor API and a gateway. (Existing figure; probe-independent.)

- Repeatability at temperature 0 is far from 1.0 for 4 of 5 endpoints; one vendor holds 1.000 over 2,800 completions incl. a 64-way burst.
- Same snapshot, two doors: gateway half as byte-stable at 64 bytes, $4\times$ less over full completions; no-seed control does not close the gap.
- No condition separates from same under the CI rule; power: a 0.10 drop is detected at 0.89–0.97 if bimodal, 0.53–0.55 if concentrated.
- Orderings differ between layers (Mistral below OpenAI on embeddings, above on generation) $\Rightarrow$ measure each layer.

6 Using ARI

Table 7: Practitioner guide.

question	run	read
Will my stored embeddings/IDs still match after a restart?	<code>proc</code>	HER_{proc} ; < 1 means re-encode or tolerate mismatches
Does traffic load change outputs?	<code>conc</code>	HER_{conc} vs HER_{same}
Will results match next week?	<code>time</code>	HER_{time} ; repeat periodically
Which provider/model drifts less?	core three	ARI-R ($1 - \bar{\rho}$), never HER
Is my self-hosted stack deterministic?	<code>proc, mach</code>	expect 1.000 at fp32, TF32 off
Did a precision/serving change happen?	<code>prec</code>	ρ jumps by orders of magnitude
Is my LLM API stable at temp 0?	ARI-D	exact-repeat rate per condition

- `pip install ari-benchmark` (pulls `semq==1.0.0`); one command per condition; report validates against the public schema; verify with the standalone verifier. **[exact commands]**
- Interpretation rules: $HER = 1.000$ means exact codes, not semantic equality or quality; compare only reports with matching inputs, probe version and conditions.

7 Limitations

- One realization per condition on APIs; CIs cover inputs, not serving states. **[weekly repeats]**
- ρ normalization verified 384–1,024-d; isotropic noise model.
- The code is a threshold detector by design: sub-threshold changes are invisible (Fig. 3); use a hash if any bit matters.
- ARI-D is a dry run: two endpoints lack `time`.
- Operator-signed reports prove integrity and origin, not neutrality.

8 Conclusion

- Two layers, one protocol, two numbers (HER for “does it match”, ρ for “how much does it move”).
- **[one-paragraph conclusion after reruns]**

References

[1] He, H. and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference. 2025. [2] Yuan, J. et al. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference. arXiv:2506.09501, 2025. [3] Rajput, S. et al. Recommender Systems with Generative Retrieval. NeurIPS 2023. [8] Yashwanth, T. S. On the Structure of Floating-Point Noise in Batch-Invariant GPU Matrix Multiplication. arXiv:2511.00025, 2025. **[carry over remaining references]**

A Probe design evidence

Table 8: Calibrated (v0.1) vs fixed-range (v0.2) probe, 8 encoders, $n_{bins} = 2$.

measure	calibrated	fixed-range
false positives	0	0
bf16 changes detected (10 cells)	88.2–99.9%	93.1–100%
Spearman(L2 drift, coordinates changed)	0.864–0.946	0.875–0.947
codes changed by recalibrating on another corpus slice	60–100%	0% (no calibration)
needs a per-model scale	yes	no

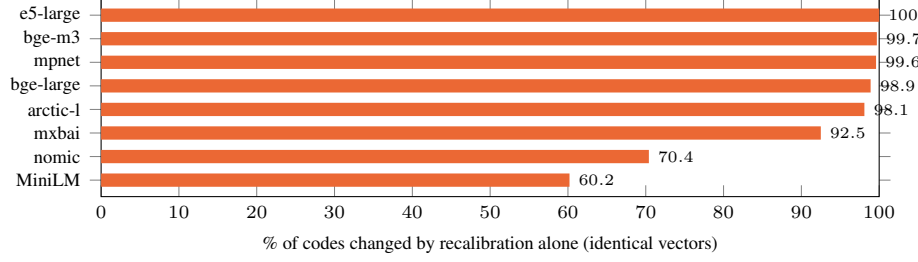


Figure 6: Calibrated probe: scale from FiQA rows vs SciFact rows of the same input set. The fixed-range probe has no such dependence.

B ARI-D-internal: the logit probe

- Centre each logit row over the vocabulary; split at fixed 65,536-wide chunks; encode each chunk with QUANT, 8 bins.
- Invariant to a constant logit shift (softmax-invariant); chunk layout fixed by vocabulary size.
- Use when internal drift below the argmax matters; a top-2 margin is a cheaper monitor for decision changes.

Table 9: Probe choice on Qwen2.5-0.5B logits (151,936 vocab, 640 rows).

	calibrated	no centring	v0.2 (centred)
detects GPU fp32 change	71.3%	76.4%	82.5%
codes changed by +3 on every logit	100%	100%	1.7%
Spearman with centred drift (bf16/int8)	0.53–0.74	0.45–0.51	0.84–0.86
Spearman with raw drift (bf16/int8)	0.96	0.57–0.67	0.54–0.70

Table 10: Decoding, 48 prompts $\times$ 48 tokens, greedy, one L40S, v0.2. bf16: <1% of tokens change, 21–42% of generations differ.

model	condition	token agreement	logit HER	generations identical
Llama-3.1-8B	proc / threads / TF32 off	100%	1.000	48/48
	TF32 on	100%	0.000	48/48
	batched (pad to 4)	100%	0.400	48/48
	fp16	99.96%	0.000	46/48
	bf16	98.90%	0.000	33/48
Qwen2.5-7B	proc / threads / TF32 off	100%	1.000	48/48
	TF32 on	99.96%	0.000	48/48
	batched (pad to 4)	100%	0.251	48/48
	fp16	99.87%	0.000	43/48
	bf16	99.03%	0.000	28/48
Mistral-7B-v0.3	proc / threads / TF32 off	100%	1.000	48/48
	TF32 on	100%	0.000	48/48
	batched (pad to 4)	100%	0.674	48/48
	fp16	99.96%	0.000	44/48
	bf16	99.38%	0.000	38/48

C Probe vs alternatives at matched storage

- Planned: code vs SHA-256, block hashes, cosine, top- k , KL/JS, top-2 margin, fp16 reference at equal bytes; 300 control + 300 intervention episodes; alarm rate fixed on controls.
- Pilot (12 prompts) is degenerate: two interventions were bit-identical, two were caught by everything, controls produced no alarms.
- **[define “material” vs “benign” change before the confirmatory run (see Fig. 3)]**

D Corrections to the v0.1 pipeline

- Model dtype not enforced (mxbai ran in fp16); now loaded in an explicit, verified dtype.
- One capture measured bf16 by in-place cast; now a fresh bf16 load.
- Self-hosted time was a same-session repeat; now a stored baseline compared ≥ 24 h later.
- A measured conc flip was recorded as 1.000; values are now recorded as measured.

E Reproducibility

- Harness Apache-2.0; spec and inputs CC BY 4.0; SEMQ SDK public (PolyForm Noncommercial 1.0.0, semq==1.0.0).
- Reports signed by the registered KMS key; standalone verifier imports neither the harness nor the SDK.
- **[commit hashes and S3/DOI for raw vectors]**